

Track A/B/C Local Pipeline To-Do

Follow these steps to move from raw frames to pruned STA evaluations. Each step lists the inputs, script to run, expected outputs, and what to inspect before proceeding.

Step 1 — Track A / Stage A: Generate Detector Proposals

- **Inputs**
 - Decision-frame images under `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg`.
 - (Optional for oracle mode) YOLO labels at `local_extraction/v2/yolo_labels_540/{clips|videos}/<uid>/<frame:07d>.txt`.
 - **What it does**
 - `trackA_stageA.py` scans frames and either replays ground-truth boxes (oracle) or runs YOLO locally to produce top-K candidate boxes per image.
 - **Command**
 - `python local_extraction/trackA/trackA_stageA/trackA_stageA.py`
 - Edit toggles in the script first: `DETECTION_MODE`, `K`, `RUN_NAME`, and YOLO weights if needed.
 - **Outputs**
 - Run folder at `local_extraction/runs/Track_A/StageA_*/*` containing `candidates.jsonl`, optional per-image CSVs, and `summary.json`.
 - **Review before continuing**
 - Open `summary.json` to confirm image counts and average boxes.
 - (If oracle mode) verify recall@K numbers when label evaluation is enabled.
-

Step 2 — Track A / Stage B Prep: Crop Candidates and Build Head Manifests

- **Inputs**
 - Stage A `candidates.jsonl` (auto-detected or set `CANDIDATES_JSONL`).
 - Source frames at `local_extraction/v2/extracted_frames/...`
 - Optional semantic manifests under `local_extraction/v2/manifests/head_{train,val}_{clip,video}.json` for verb/noun/TTC labels.
- **What it does**
 - `trackA_stageB.py` crops ROIs for each candidate, measures recall (optional), and emits Stage B head manifests (`head_train.jsonl`, `head_val.jsonl`) aligned with the crops.
- **Command**
 - `python local_extraction/trackA/trackA_stageB/trackA_stageB.py`
 - Adjust toggles such as `KEEP_TOP_N`, `CROP_SIZE`, manifest paths, and run directory name.
- **Outputs**
 - Run directory `local_extraction/runs/Track_A/trackA_stageB_*/*` with:
 - `crops/` (per-candidate images).
 - `manifest.jsonl` describing each crop (UID, frame, box, paths).

- `head_train.jsonl` / `head_val.jsonl` when semantic manifests are available.
 - Updated Stage A `summary.json` with recall metrics if enabled.
 - **Review before continuing**
 - Check `head_train.jsonl/head_val.jsonl` line counts to ensure candidates were matched.
 - Spot-check a crop in `crops/` to verify bounding boxes look correct.
-

Step 3 — Track B: Train Lightweight Fusion Head

- **Inputs**
 - Stage B manifests from Step 2 (`head_train.jsonl`, `head_val.jsonl`).
 - Extracted frames / clips already on disk (no new data extraction required).
 - **What it does**
 - `trackB_train_loader.py` tokenizes image and clip grids, applies FGTP + dual cross-attention, ROI-pools per candidate, and trains the STA head (next-active, verb, TTC).
 - **Command**
 - `python local_extraction/trackB/trackB_train_loader.py`
 - Tune configs in the script (`TrainConfig` block) for batch size, epochs, evaluation cadence, and manifest overrides.
 - **Outputs**
 - `local_extraction/runs/Track_B/cache/` with dataset stats and invalid record logs.
 - `local_extraction/runs/Track_B/checkpoints/` containing per-epoch, final, and best checkpoints (e.g., `trackB_final_<timestamp>.pt`).
 - Validation metrics/predictions/overlays in `runs/Track_B/{metrics,predictions,overlays/val}`.
 - **Review before continuing**
 - Inspect `metrics/metrics_val_<timestamp>.json` for accuracy, N mAP, and TTC MAE.
 - Open a few overlay images to confirm qualitative behavior.
-

Step 4 — Track B: Standalone Evaluation (Optional Re-Run)

- **Inputs**
 - Latest Track B checkpoint and validation manifest (auto-discovered if left `None`).
 - **What it does**
 - Recomputes metrics and overlays without retraining, useful after adjusting configs or to double-check best checkpoints.
 - **Command**
 - `python local_extraction/trackB/trackB_eval.py`
 - **Outputs**
 - Fresh metrics JSON, candidate-level predictions, and overlays under `runs/Track_B/` with new timestamps.
 - **Review before continuing**
 - Confirm metric stability and that the intended checkpoint was loaded (check console output).
-

Step 5 — Track C: Apply RGTP Pruning at Inference

- **Inputs**
 - Track B checkpoint (best or final) and associated Stage B run directory.
 - Validation manifest (auto-detected by default).
 - **What it does**
 - `trackC_pruning.py` loads Track B, applies rollout-guided token pruning (controlled by `rgtp_rate`), and measures accuracy/mAP/TTC with pruning statistics.
 - **Command**
 - `python local_extraction/trackC/trackC_pruning.py`
 - Edit `RuntimeConfig` for checkpoint path, pruning rate, and toggles (`pruning_enabled`, `min_keep`).
 - **Outputs**
 - Metrics JSON at `local_extraction/runs/Track_C/metrics/trackC_val_rateXX_<timestamp>.json` summarizing accuracy, N mAP, TTC MAE, and mean prune fraction.
 - **Review before concluding**
 - Compare metrics against the unpruned baseline to document latency vs. quality trade-offs.
 - Note the prune rate that keeps mAP drop ≤ 0.5 while improving efficiency.
-

Final Conclusion

Once Step 5 metrics are satisfactory, assemble the following artifacts for reporting:

- Stage A summary (`runs/Track_A/.../summary.json`) capturing proposal recall and settings.
- Stage B best checkpoint + validation metrics/overlays demonstrating performance.
- Track C pruning report showing accuracy vs. prune rate trade-offs.

Review these side by side to confirm the pipeline achieves the desired recall-first detection, lightweight fusion gains, and pruning-driven efficiency before integrating results into the thesis or deployment notes.